Learning Orbital Rendezvous

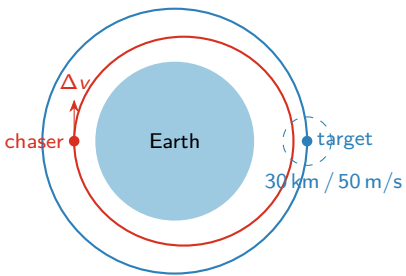
A reinforcement-learned guidance policy for a chaser spacecraft

Peter Brown

October 2026

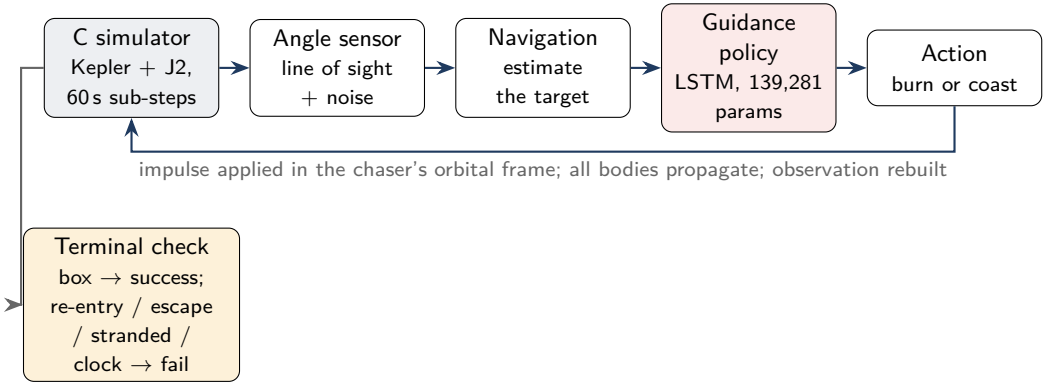
One policy, seven rendezvous skills in 3-D, 96–99.5 % held-out success on the five wide-box skills.
Simulator in C, policy in PyTorch, every number measured on held-out episodes.

The problem: get a chaser into a box around a passive target



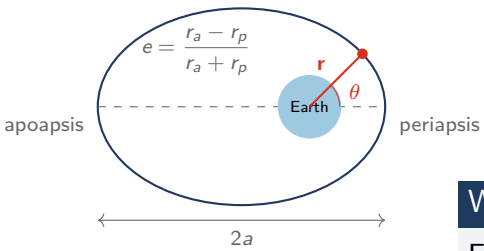
- **Success** inside 30 km and 50 m/s of the target
- **Failure** re-enter, escape, run dry, or run out of clock
- **Tank** 478 m/s (15 % propellant); the generalist draws 353–656 m/s per episode
- **Start** random orbits, phase gap up to $\pm 180°$; LEO 300–800 km in the headline, up to 8,000 km and $e \leq 0.3$ in the generalist
- **Act** a fixed-size burn, or a coast of 1 min to 24 h

One loop, every decision



- The policy sees 38 numbers and picks one of 31 actions. It never sees the target's true state in the angles-only configuration.
- The same loop runs in allocation-free C, action for action identical to PyTorch.

Simulator: analytic two-body plus J2, written in C

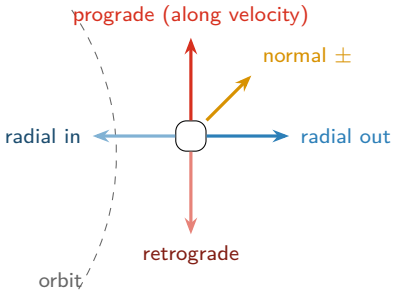


- **State** orbital elements $(a, e, i, \Omega, \omega, M)$
- **Propagate** Kepler's equation, closed form at any step
- **Burns** impulsive, in the chaser's own frame
- **J2** Earth's bulge slowly rotates the orbit plane

Why analytic

Exact at any step size, so a 24-hour coast is one solve. About half a million environment steps per second on one laptop CPU.

Actions: a short menu of fixed burns and coasts



burn grades 1, 2, 5, 10, 25 m/s

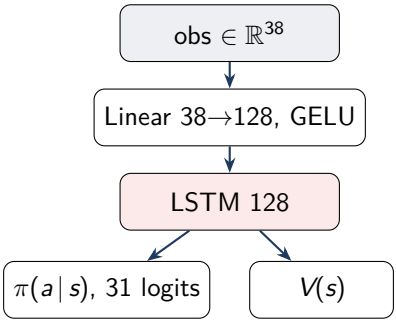
Coasts: no burn, physics and sensor still run every 60 s



Each decision lasts as long as the coast it chose; a burn takes one sub-step. 31 actions in all.

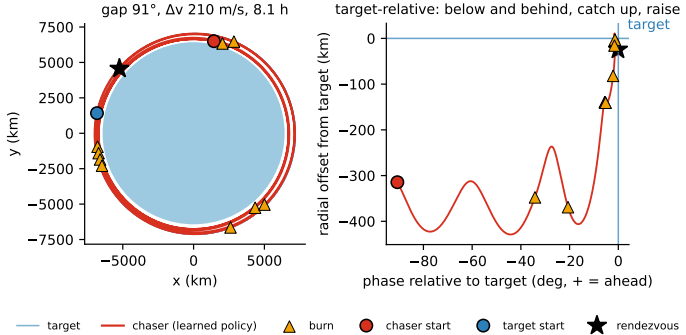
A rendezvous is hours of waiting and seconds of burning. Letting the policy pick *how long* to wait is what makes it learnable.

Policy and training



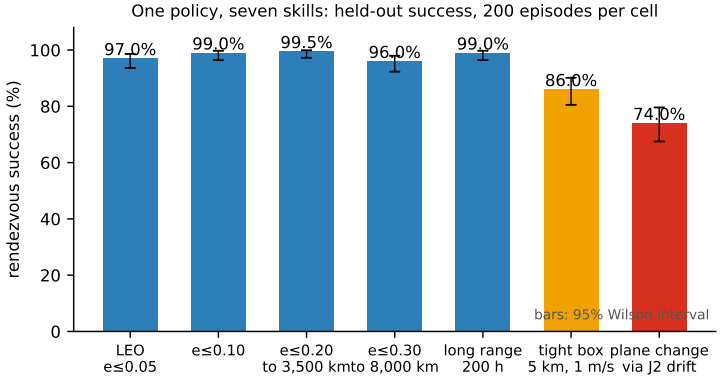
- **Small and recurrent** 139,281 parameters; the LSTM carries what the last burn did
- **PPO** 1024 parallel environments, 50 M steps per stage, about two hours each
- **Reward** $+1$ success, -1 failure, with potential-based shaping in between
- **Curriculum** one new difficulty per stage, warm-started

What it learned, 2-D: phasing, shape matching, and when to wait



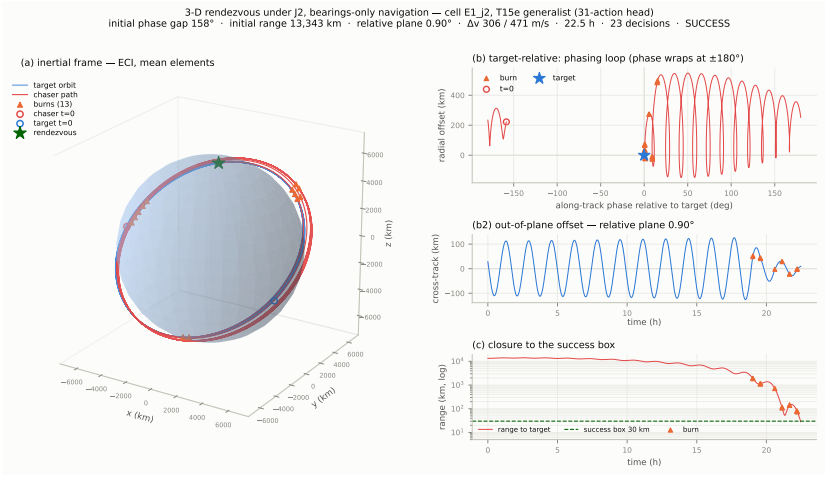
- **200/200** held-out episodes on **5/5 training seeds**; wide envelope (300–8,000 km, $e \leq 0.30$) 800/800 on 4 seeds.
- Fuel: median 1.18× a two-impulse optimum on the same clock (appendix A4).
- Phasing by altitude, apse and eccentricity matching, and coasting through windows where a burn would not help.

One policy, seven rendezvous skills, in 3-D with J2



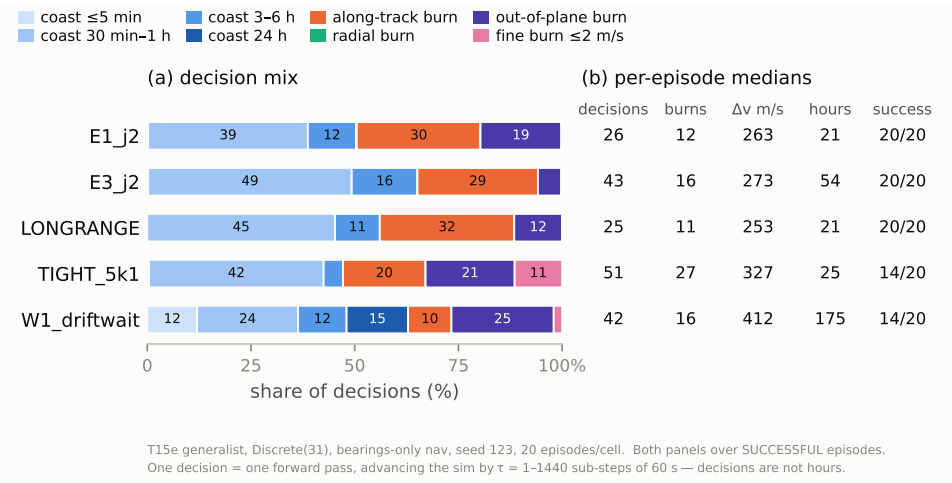
Same network, same weights, every cell. 200 held-out episodes per skill, flying on the angles-only estimate.

The same policy flying one 3-D episode



A 158° gap and a 0.9° plane difference closed in 22.5 h with 306 m/s and 23 decisions, on the filter's estimate of the target.

What the policy actually does, scenario by scenario



Decision mix of the seven-skill policy on five of its scenarios, 20 held-out episodes each (success rates on slide 8 are the 200-episode ones). The same weights coast a day at a time only when a plane gap can be drifted shut, use fine 1–2 m/s burns only for the tight box, and never touch the radial burns.

What is open, and what I would do next

Open

- The seven-skill policy is one training seed
- Fuel is 1.18× optimal; the burn grid is coarse
- Chaser navigation assumed perfect; no eclipses or sensor bias
- Two-body plus secular J2 only

Next

- Finer burn grades
- More seeds on the generalist
- A maneuvering target

Appendix

observation vector · phasing and J2 · the seven cells · fuel audit · Lambert search · navigation in one picture · C port · how the seventh skill was trained · FAQ

A1 Observation vector (38 floats, roughly in $[-1, 1]$)

slots	value	why
0–1	chaser altitude / scale, e	size and shape
2–3	$\sin\theta, \cos\theta$	position on orbit, no wrap
4–5	v_r/v_{circ}, v_t/v_{circ}	radial and tangential speed
6	fuel fraction	budget remaining
7–8	target altitude, e (estimated)	target size and shape
9–12	$\sin, \cos\ \omega$ for both	apse orientations
13–14	$\sin, \cos\ \Delta\lambda$ (mean-longitude gap)	the phase gap
15	(cap − step)/cap	episode clock
16	$\cos(\omega_s - \omega_t)$	apsidal alignment
17–20	nearest body: distance, bearing, closing rate, keep-out	Earth
21–25	relative-inclination vector; $\Delta\mathbf{e} \cdot \hat{h}_t$; cross-track offset and rate	3-D block
26–28	Δv_{plane}, $\Delta v_{remaining}$, feasibility margin	fuel ledger
29–30	$\cos i_s$, $\cos i_t$	J2
33–36	LVLH radial / along-track offset; relative velocity, rotating-frame corrected	relative state
37	target mean motion	

Target quantities are the filter's estimate in bearings-only mode. The policy is not given the covariance (tested: a σ channel added nothing).

A2 Phasing, and plane change by burning versus by J2

Phasing $T = 2\pi\sqrt{a^3/\mu}$; a chaser 100 km below a LEO target gains about 4° of phase per orbit, so "drop, wait N orbits, raise" is the textbook catch-up and it is what the policy does.

Plane change by burning $\Delta v = 2v\sin(\Delta i/2)$. At 7.5 km/s a 3° change costs about 390 m/s, most of a 478 m/s tank.

Plane change by J2 Earth's equatorial bulge makes the node line drift around the pole at

$$\dot{\Omega} = -\tfrac{3}{2}\, J_2\, n \left(\frac{R_E}{p}\right)^2 \cos i, \qquad p = a(1-e^2).$$

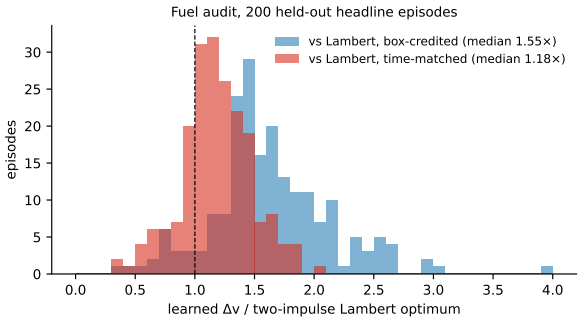
The rate depends on altitude and inclination, so two orbits at different altitudes drift apart in Ω at a controllable relative rate. Dip low, coast a day at a time, let the node drift close the plane gap, then raise. In the specialist about half of the plane alignment was bought this way.

A3 The seven cells

cell	altitude band (km)	e_{max}	box	clock (sub-steps)	success
E0_j2	300–800	0.05	30 km / 50 m/s	3,000	97.0 %
E1_j2	300–1,500	0.10	30 km / 50 m/s	3,000	99.0 %
E2_j2	300–3,500	0.20	30 km / 50 m/s	4,500	99.5 %
E3_j2	300–8,000	0.30	30 km / 50 m/s	6,000	96.0 %
LONGRANGE	300–3,500	0.12	30 km / 50 m/s	12,000	99.0 %
TIGHT_5k1	300–800	0.05	5 km / 1 m/s	3,000	86.0 %[†]
W1_driftwait	300–800	0.05	30 km / 50 m/s	22,000	74.0 %

[†]92.5 % at the training fuel floor. All cells: 3-D, J2 on, targets inclined 30–60°, relative plane up to 0.75–1° (2–5° for W1), bearings-only navigation, Discrete(31) head, fuel budget sampled per episode (353–656 m/s). Held-out seed 123, 200 episodes per cell, greedy actions, LSTM state zeroed per episode.

A4 Fuel audit against a two-impulse optimum



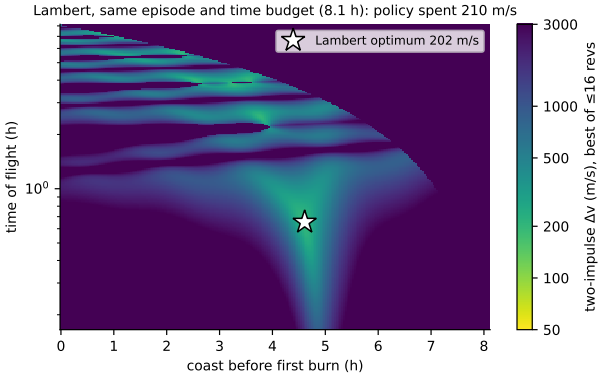
For each episode an offline solver finds the cheapest two-burn transfer from the chaser's start to the target, searching over how long to wait before the first burn and how long to fly, under the same clock the policy used.

Time-matched must hit the target's exact position and velocity: median learned Δv is **1.18**× it.

Box-credited the same solution minus the 50 m/s velocity tolerance the policy is never asked to close: **1.55**×. The fairer of the two.

200 held-out episodes. Ratios below 1 happen because arrival is credited inside a box and a two-burn plan is not the unconstrained optimum.

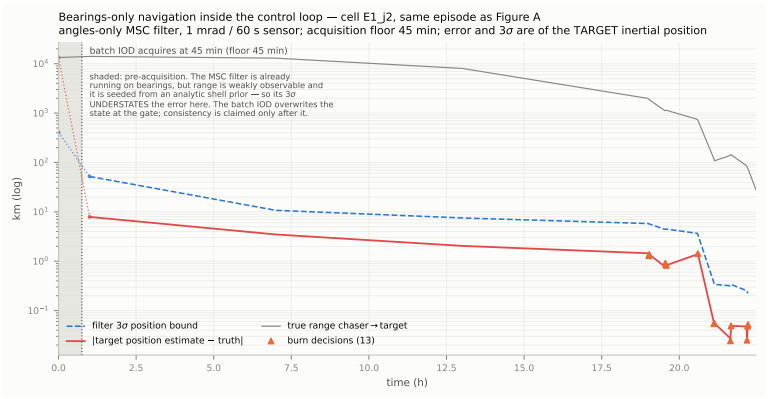
A5 What the Lambert search looks like for one episode



Every cell is one candidate plan: wait this long, then fly for that long. Colour is the two-burn Δv of the best conic connecting the two points in that time, including multi-revolution solutions. The banded structure is the orbital period showing through: cheap plans recur once per orbit.

The star is the cheapest plan inside the policy's own clock. On this episode the optimum was 202 m/s and the policy spent 210.

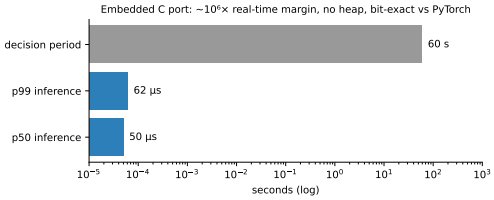
A6 Navigation in one picture: blind, acquire, converge



The chaser measures only the direction to the target, not the distance. One direction tells you nothing about range; a sequence of directions while both orbits curve does. A batch solver tries many ranges along the first line of sight, propagates each, and keeps the one whose predicted directions match what was seen. That hands a full state to a recursive filter that keeps refining it.

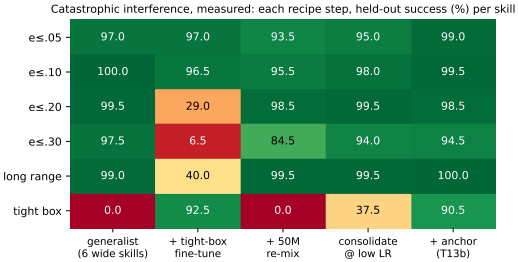
Policy result: trained on truth and handed the estimate, **51 %**; trained with the filter in the loop, **97 %** (2-D tight box) and **46→98 %** across three seeds in 3-D. It learned to wait for acquisition and burn nothing while blind (median Δv before acquisition 75→0 m/s).

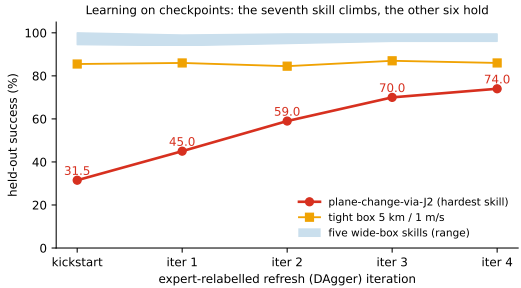
A7 C port of the policy



- The trained weights exported to a C header; forward pass written by hand: linear, GELU, LSTM cell, two linear heads.
- No heap, no globals; the caller owns the 1 kB of recurrent state; fixed work per call.
- Checked against PyTorch: identical action on 7,048 recorded observations and an identical action stream over 200 closed-loop episodes.
- Median 50 μs per decision on a laptop against a 60 s decision period (a benchmark, not a worst-case timing).

A8 How the seventh skill was trained





- Left: fine-tuning on one skill damaged others; a low learning rate plus replay of each skill's own trajectories held all six.
- Right: the plane-change skill never bootstrapped from reward. A scripted expert demonstrated it, the policy was refreshed on the expert's corrections four times, and each refresh was scored on 200 held-out episodes.

A9 FAQ

Why RL, when Lambert exists? Lambert needs perfect state and gives an open-loop two-burn plan; the policy is closed-loop on an estimate, respects a budget, and one network covers seven regimes. Lambert is the grader, not the competitor.

Why discrete actions? PPO is stable with them; a continuous head collapsed early on. The grid costs fuel (1.14–1.18× across two evaluation sets); finer grades are the obvious next step.

Is 200 episodes enough? At 97 % the 95 % Wilson interval is about ±2.5 pp; for 0/200 failures the lower bound is 98.2 %. I report intervals, not "100 %".

Single seed? The generalist, yes. The 2-D headline (5), wide envelope (4), 3-D nav and J2 rungs (3) are multi-seed.

Why coast 24 h? J2 nodal precession closes a plane gap for free; burning it would exceed the tank. The policy learned to wait.

How does the filter work? Angles only, so range comes from how the bearing changes while the orbits curve; a batch solver picks the range that explains the first hour of bearings, then a recursive filter refines it. The finding I own is on the policy side: train on the estimate or lose 46 points.

What would you do differently? Audit against physics from day one; log learning curves on every run; design the observation with the filter in mind from the start.